Thamill Indian B U
Cohort-15 (AI Engineer)

Day 13 Post-Session Assignment & Engineering Reflection

AI Engineering Program | Day 13: Hypothesize a Production-Ready AI Solution
Repository: https://github.com/ThamillIndianBlend360/AI-Assignments/tree/main/Day13

Part 1: Hypothetical Agentic System Design Card

This design card establishes the engineering blueprint for taking the Enterprise HR Leave Assistant prototype (developed in Days 10–12) into a production-ready enterprise service. It leverages the Blend360 Agentic DevOps Loop principles, enforcing strict contract handoffs and role isolation.

Current Prototype Baseline	[EXISTING / OBSERVED] • Local Python backend running an Ollama (llama3) multi-turn ReAct loop. • Pydantic schema validation for tool arguments (LeaveBalanceInput, LeaveRequestInput). • Programmatic balance enforcement and idempotency key caching ((user_id, date)). • Execution audit logging saved to day12_audit_trail.log. • Terminal-based interactive confirmation prompt for write operations.
Proposed Architecture & Workflow	[HYPOTHESIZED / PROPOSED] • API Gateway & Auth: OAuth2/SAML SSO issuing JWTs with employee ID & role scope. • Stateless Orchestrator: FastAPI application hosting the LLM tool parser. • State Store & Locking: PostgreSQL database with row-level locks for leave records. • Governance Gate: Asynchronous webhook approval service for manager authorizations.
Assumptions & Customer Unknowns	[NEEDS VALIDATION / HYPOTHESIZED] 1. Employee identity can be securely mapped from OAuth claims to employee IDs. 2. Downstream HRIS systems expose REST endpoints supporting idempotency headers. 3. Average leave request volume is <5,000 requests/day with peak concurrency <50 req/sec.
Lifecycle Handoff Contracts	[HYPOTHESIZED LIFECYCLE] • Discover -> Plan: Signed SRS containing 12 functional requirements & RBAC matrix. • Build -> Verify: Clean Pydantic compilation and 100% pass on unit test suite. • Verify -> UAT: Staging deployment with golden set accuracy >= 98%. • Release -> Roll Out: Version-pinned container bundle with rollback script tested.

Pilot Go/No-Go Conditions	[HYPOTHESIZED THRESHOLDS] • Zero unhandled exceptions or state corruptions over 100 consecutive automated test runs. • 100% enforcement of human approval gates on state-mutating requests. • Latency under 2.5 seconds for end-to-end tool parsing and execution.
Design Alternatives & Trade-offs	[CHOSEN: Deterministic Hybrid Workflow | REJECTED: Fully Autonomous Agent] • Trade-off: Higher upfront engineering effort for Pydantic schemas and Python guardrails in exchange for 100% deterministic mathematical safety and zero unauthenticated write mutations.

Part 2: Written Reflection

1. Current State vs. Future State

What have you actually built or tested? What production capabilities are only proposed?
• **[EXISTING / OBSERVED]:** I have built and locally verified a working multi-turn ReAct agent in Python (`Day12/main.py`) powered by a local Ollama (`llama3`) model. The prototype includes strict Pydantic argument validation, in-memory idempotency caching to block duplicate leave requests, deterministic code-level leave balance comparisons, structured audit trail logging (`day12_audit_trail.log`), and an interactive terminal confirmation gate for write actions.
• **[HYPOTHESIZED / PROPOSED]:** Production capabilities that exist only as proposals include: multi-tenant cloud deployment on FastAPI/Docker, OAuth2/JWT user identity propagation, persistent PostgreSQL database storage with row-level transactional locks, asynchronous Slack/Teams webhook notifications for manager approval, and automated telemetry tracking via OpenTelemetry/Prometheus.

2. Key Hypotheses

What are your three most important assumptions about users, data, system behavior, or operating constraints? How would you validate each?

1. Hypothesis 1 (System Behavior / Idempotency): Generating unique idempotency keys based on `(user_id, date, leave_type)` in the application layer will eliminate 100% of duplicate leave requests caused by network retries or LLM loop re-entry.
• *Validation Plan:* Run a multi-threaded stress test simulating 50 concurrent identical requests and verify that only 1 database entry is created while 49 return cached replay responses.

2. Hypothesis 2 (Input Validation / Guardrails): Strict Pydantic schema validation will intercept at least 95% of malformed LLM tool call arguments before backend tool execution.
• *Validation Plan:* Inject 100 corrupted tool JSON payloads (e.g., negative days, string dates, missing keys) and verify that 0 corrupted payloads reach the tool execution function.

3. Hypothesis 3 (User & Data Constraints): Users will accept an explicit confirmation step for write actions without abandoning the conversation.
• *Validation Plan:* Conduct a pilot user test tracking completion rates when confirmation prompts are required versus bypassed.

Thamill Indian B U
Cohort-15 (AI Engineer)

3. Autonomy and Safety Boundaries

What decisions would the model make? Which rules belong in deterministic code? Which actions need approval, and why?

• **LLM Model Decisions:** The LLM is strictly restricted to natural language understanding, user intent classification, slot-filling (extracting user ID, dates, and festival names), and formatting user-facing natural language responses.
• **Deterministic Code Rules:** Mathematical balance comparisons (e.g., `remaining_balance - requested_days >= 0`), date parsing/validation, user authentication/authorization lookups, idempotency caching, and transaction logging reside entirely in Python backend code.
• **Human Approval Gates:** Any state-mutating tool (`create_leave_request`, `cancel_leave_request`) REQUIRES explicit human approval. The model can only propose a mutation; the Python execution engine intercepts the proposal and pauses execution until authorized. This prevents unauthorized database mutations caused by prompt injection or model hallucinations.

4. Failure Thinking

Describe one realistic production failure. What should the system do, who responds, and what evidence would reveal the problem?

• **Scenario:** The downstream Enterprise HRIS API times out mid-transaction after the agent has approved a leave request, causing a network disconnect.
• **Expected System Behavior:** The Python execution layer catches the backend API timeout exception, aborts the transaction, prevents balance deduction, and logs an `HRIS_API_TIMEOUT` error event. The agent informs the user: 'The HR system is currently unreachable. Your request was not submitted and your balance remains unchanged. Please try again later.'
• **Responding Party:** On-call DevOps / Platform Reliability Engineer.
• **Revealing Evidence:** The structured audit trail log (`day12_audit_trail.log`) records an attempted tool call, followed by an exception trace with `status: ERROR` and `error_code: HRIS_API_TIMEOUT`, while the database state shows no newly committed record.

5. Evaluation Design (Golden Test Cases)

For each golden case, state input, expected outcome, evaluation criteria, and how you would test it. Mark any real observations separately.

Case Type	Input / Scenario	Expected Outcome	Evaluation Criteria	Validation Method & Status
Normal Path	User E101 (balance 3) requests 1 day leave for Diwali (2026-11-08).	Balance checked first (3), leave request created, balance reduced to 2.	Tool sequence: get_balance -> create_request. Exact status == SUCCESS.	[OBSERVED REAL TEST] Passed in Day12 test suite. Result logged in day12_audit_trail.log.
Dependency Failure	User E101 submits request while HRIS API returns 504 Gateway Timeout.	System catches exception, rolls back state, returns user-friendly error.	No DB state mutation; audit log captures HRIS_TIMEOUT error code.	[HYPOTHESIZED TEST] Mock unit test throwing requests.exceptions.Timeout exception.
Access / Security Boundary	User E102 (balance 0) attempts to request 2 days leave.	Deterministic code blocks request due to insufficient balance; 0 leave created.	Tool call rejected with error_code: INSUFFICIENT_BALANCE.	[OBSERVED REAL TEST] Passed in Day12 test suite. Balance check blocked request.

6. Trade-offs and Alternatives

What alternative architecture or workflow did you consider? Compare reliability, latency, cost, security, complexity, and user value.

Dimension	Chosen Architecture: Deterministic Hybrid Agent	Rejected Alternative: Fully Autonomous LLM Agent
Reliability	100% deterministic rule enforcement in Python code.	Unreliable; subject to model hallucinations and prompt drift.
Latency	Slightly higher (2 turns + confirmation gate).	Lower latency per turn, but high retry rate on tool failure.
Cost	Low; minimal token usage with explicit JSON schemas.	High; longer prompts and frequent multi-turn retries.
Security	High; application-level RBAC & human approval gate.	Critical risk; direct DB access exposes vulnerability to prompt injection.

Complexity	Moderate; requires Pydantic models & validation code.	Low initial code complexity, but high operational maintenance.
User Value	High trust; reliable leave booking with zero math errors.	Low trust; risk of accidental leave deductions or errors.

7. Pilot Readiness Decision

What conditions must be met before a limited pilot? Which unknowns are blockers, and which can be monitored during a pilot?

- **Mandatory Go/No-Go Conditions Before Pilot:**
  1. 100% pass rate on the 12-case golden evaluation test suite over 100 consecutive runs.
  2. Zero unhandled exceptions or state corruptions during simulated API network latency.
  3. Security sign-off on OAuth2 identity propagation and human confirmation gate implementation.
- **Blocker Unknowns (Must resolve BEFORE pilot):**
  - HRIS API authentication and transaction rollback capabilities.
  - Data privacy compliance for employee HR records.
- **Monitorable Unknowns (Can evaluate DURING pilot):**
  - Real-world prompt variations and natural language edge cases.
  - End-to-end response latency under live employee concurrency.

8. Next Experiment

What is the smallest hands-on test or prototype enhancement that would reduce the biggest uncertainty?

- **Experiment Title:** Automated Golden Regression Test Harness (`Day13/test_harness.py`).
- **Objective:** Build a lightweight script that automatically runs the local Ollama ReAct agent against a dataset of 15 structured scenarios (valid requests, insufficient balances, ambiguous dates, unknown festivals, and malformed inputs) after every code or prompt modification.
- **Learning Goal:** Quantify model consistency, tool selection accuracy, and schema validation reliability across repeated local runs, proving that prompt updates do not introduce silent regressions.